



Tecnológico de Monterrey

Estudiantes:

Axel Javier Rosas Rodríguez | A01738607

Elian Cantalapiedra Sabugal | A01738462

Edwin Emmanuel Salazar Meza | A01738380

Profesor:

Rigoberto Cerino Jiménez

Nombre de la Institución:

Instituto Tecnológico y de Estudios Superiores de
Tecnológico de Monterrey

Materia:

Implementación de Robótica Inteligente

Fecha:

11 de mayo de 2026

Título del trabajo:

Reto semanal 4. Manchester Robotics

Resumen

Este reporte documenta el desarrollo e implementación de un sistema de navegación autónoma para un robot móvil diferencial tipo Puzzlebot, capaz de realizar navegación punto a punto mientras integra una capa de toma de decisiones basada en visión computacional. El proceso incluyó la corrección de la odometría, la sintonización de un controlador robusto Proporcional-Integral (PI) para operar en superficies de baja fricción y el diseño de un algoritmo de procesamiento de imágenes para la detección de semáforos. Los resultados demuestran un sistema capaz de modular su velocidad en tiempo real (deteniéndose en rojo, ralentizando en amarillo y avanzando en verde) respondiendo eficazmente a estímulos visuales externos.

Objetivos

Objetivo General:

- Implementar una capa de toma de decisiones en un algoritmo de navegación punto a punto para detectar y responder a los colores de un semáforo.

Objetivos específicos:

- Desarrollar un sistema de visión robusto que identifique círculos y segmente colores (rojo, amarillo, verde) en el espacio HSV.
- Diseñar y sintonizar un controlador de lazo cerrado que considere perturbaciones, ruidos en la odometría y no linealidades.
- Garantizar que el robot mantenga el estado de "detenido" ante una luz roja, incluso si esta deja de ser detectada, hasta recibir una señal verde.

Introducción

La robótica móvil autónoma requiere la integración de sistemas de percepción y control para interactuar con entornos dinámicos. En este contexto, el Controlador PI (Proporcional-Integral) aplicado a robots diferenciales permite minimizar el error de estado estable comparando las velocidades deseadas con las reales estimadas por odometría. La robustez de este controlador se define como su capacidad para absorber perturbaciones, como el deslizamiento en superficies lisas o el ruido en los encoders. Para la percepción, se utiliza una tarjeta embebida (Jetson Nano) interconectada con una cámara mediante ROS (Robot Operating System).

El procesamiento de imágenes emplea visión clásica: se segmenta la imagen mediante el espacio de color HSV (Matiz, Saturación, Valor), el cual es más robusto ante cambios de iluminación que el espacio RGB. Los métodos utilizados incluyen la detección de contornos y formas geométricas para identificar la estructura circular del semáforo, aplicando filtros de área para evitar falsos positivos por ruido visual.

Metodología

1. Estrategia de Solución

El problema central de este reto consistió en integrar al algoritmo de navegación punto a punto una capa de visión computacional para la toma de decisiones. La estrategia de solución se dividió en tres tareas principales:

- Procesamiento de imagen: Detección de colores (rojo, amarillo, verde) de un semáforo usando vision computacional.
- Control robusto: Mejora del controlador de lazo cerrado para garantizar que la navegación al punto objetivo sea precisa, considerando perturbaciones y ruido en la odometría.
- Unificación del control y estado del semáforo: Integración de los comandos de navegación con la capa de decisión visual para modular el comportamiento del robot en tiempo real.

2. Procesamiento de Imagen y Detección del Semáforo

- Detección de forma: En lugar de buscar directamente parches de color en toda la imagen, el algoritmo escanea primero la presencia de figuras geométricas circulares. Esto permite identificar la estructura física de las luces de un semáforo.
- Validación por área: Para descartar ruido visual o círculos irrelevantes en el fondo, se aplica un filtro de tamaño. El sistema evalúa el contorno detectado y determina que un objetivo solo es válido si su área se encuentra estrictamente dentro de un umbral de 170 a 1000 píxeles.
- Identificación de color: Una vez que el círculo objetivo está validado, se procesa esa región específica de la imagen utilizando el espacio de color HSV (Matiz, Saturación, Valor). Dependiendo del rango en el que caigan los valores detectados, el sistema etiqueta el semáforo como Rojo, Amarillo o Verde y publica este estado.

3. Robustez, Mejoras del Control y Sintonización

- Corrección de Odometría: Se revisó y corrigió el cálculo de la odometría, asegurando una estimación de la postura del robot mucho más fiel a la realidad y reduciendo el error acumulativo provocado por la cinemática.
- Ajuste por Baja Fricción y Cálculo de Trayectorias: Al operar sobre una superficie de baja fricción, el robot era susceptible a un mayor deslizamiento y pérdida de tracción. Para mitigar esto, se modificó el cálculo de trayectorias para ejecutar los movimientos de giro y avance de manera simultánea, pero buscando corregir siempre primero la orientación. Esta asegura que el robot apunte hacia el objetivo antes de aplicar velocidades lineales altas, evitando desviaciones descontroladas.

- Sintonización: Debido a la nueva lógica de trayectorias simultáneas y a las condiciones de la superficie, fue necesario realizar un reajuste de los parámetros de control: Se ajustaron nuevamente los valores de las ganancias proporcional (K_p) e integral (K_i), haciéndolo de forma independiente tanto para la velocidad angular como para la velocidad lineal.

3. Unificación del Control y Estado del Semáforo

El nodo de `path_generator` calcula continuamente las velocidades necesarias (lineal y angular) para alcanzar la coordenada objetivo, basándose en la retroalimentación de la odometría. Simultáneamente, el controlador lee el tópico correspondiente al estado del semáforo y aplica la siguiente lógica matemática sobre la velocidad calculada:

- Verde: La velocidad calculada se multiplica por un factor de 1. El robot avanza hacia el objetivo con su rendimiento nominal.
- Amarillo: La velocidad calculada se multiplica por un factor de 0.2. Esto aplica una restricción inmediata en el comando de salida, obligando al robot a conducir de forma lenta y precavida sin perder su trayectoria.
- Rojo: La velocidad calculada se multiplica por un factor de 0. Las señales enviadas a los motores se anulan por completo, logrando que el robot se detenga y espere en su posición actual hasta que la luz cambie

Solución del problema

Odometry_robot

Este nodo calcula la posición física en el espacio dónde se encuentra el robot basándose únicamente en qué tan rápido giran sus ruedas. Para ello partimos de una configuración inicial:

- Constantes físicas (l , r): l es la distancia que hay entre las dos ruedas (el ancho del eje), y r es el radio de sus llantas. Sin estas medidas exactas, el robot calcularía mal sus distancias.
- Filtro de suavizado ($\alpha = 0.85$): Los sensores de los motores suelen mandar lecturas ruidosas (ruido eléctrico). Esta variable sirve para crear un filtro que suaviza esos saltos.
- Memoria de Posición (X , Y , θ): Aquí el robot anota sus coordenadas actuales. Inicia creyendo que está en el punto (0, 0) mirando hacia el ángulo 0.
- Reloj Interno (`start_time`, `last_time`): Variables para medir el tiempo exacto que pasa entre un cálculo y otro.

```
#parameters of the system
self._l = 0.173
self._r = 0.054
self._sample_time = 0.01
self.rate = 100.0 #Hz

#Variables to be used
self.v_r = 0.0
self.v_l = 0.0
self.V = 0.0
self.Omega = 0.0

#Messages to be used
self.alpha = 0.85
self.wr = Float32()
self.wl = Float32()
self.wr_f = 0.0
self.wl_f = 0.0
self.odom_msg = Odometry()

#Pose del robot
self.X = 0.0
self.Y = 0.0
self.Th = 0.0
```

Imagen 1. Condiciones iniciales y definición de variables

El robot se encuentra constantemente recibiendo información de los sensores de sus ruedas (los encoders), que le dicen a qué velocidad angular giran las ruedas izquierda y derecha (wl y wr). En lugar de tomar esta lectura, aplica un pequeño filtro matemático, multiplica la lectura nueva por un 85% (alpha) y le suma el 15% de la lectura anterior. Esto crea una aceleración y desaceleración suave en sus datos, evitando que un pico de ruido arruine el cálculo de su posición.

```
# Callbacks
def encR_callback(self, msg):
    self.wr = msg
    self.wr_f = (self.alpha * self.wr.data) + (1.0 - self.alpha) * self.wr_f
    #self.wr_f = self.wr.data
    self.get_logger().info(f'Vel Derecha: {self.wr_f}')

def encL_callback(self, msg):
    self.wl = msg
    self.wl_f = (self.alpha * self.wl.data) + (1.0 - self.alpha) * self.wl_f
    #self.wl_f = self.wl.data
    self.get_logger().info(f'Vel Izquierda: {self.wl_f}')
```

Imagen 2. Lectura de Encoders

Por último, el nodo mide el tiempo que acaba de pasar dt y convierte el giro de cada motor en velocidad física (metros por segundo). Con eso, calcula el movimiento de todo su cuerpo y sabiendo a qué velocidad va y hacia dónde rota, calcula en dónde quedó en ese instante dt y lo suma a su posición anterior. Toma la distancia total que avanzó y usa trigonometría para saber qué porcentaje de ese avance fue en el eje X y cuánto en el eje Y.

```

#Wheel Tangential Velocities
self.v_r = self._r * self.wr_f
self.v_l = self._r * self.wl_f

#Robot Velocities
self.V = (0.5) * (self.v_r + self.v_l)
self.Omega = (1.0 / self._l) * (self.v_r - self.v_l)

self.Th += self.Omega * dt
self.Th = np.arctan2(np.sin(self.Th), np.cos(self.Th))
self.X += self.V * np.cos(self.Th) * dt
self.Y += self.V * np.sin(self.Th) * dt

```

Imagen 3. Cálculo numérico de la odometría

Path_Generator

Se encarga de calcular las velocidades lineales y angulares del Puzzlebot para alcanzar su trayectoria objetivo, para esto considera la diferencia del error respecto a la posición actual. Asimismo, se encarga de leer el estado del semáforo y dependiendo el color modifica las velocidades generadas.

Para empezar comenzamos definiendo las coordenadas (x, y) que debe recorrer el Puzzlebot, junto a los límites físicos de las velocidades (max_linear_vel, max_angular_vel) además de una pequeña tolerancia, donde si el robot llega a esta distancia, asume que ha alcanzado el objetivo y puede pasar al siguiente punto.

También se define brevemente la lógica de Semáforo, asignando un factor multiplicador según el color identificado.

```

# Lista de objetivos (x, y)
self.targets = [
    {'x': 2.0, 'y': 0.0},
    {'x': 2.0, 'y': 2.0},
    {'x': 0.0, 'y': 2.0},
    {'x': 0.0, 'y': 0.0}
]

```

```

# Parámetros de límites físicos
self.max_linear_vel = 0.4
self.max_angular_vel = 4.0
self.dist_tolerance = 0.05

# Lógica de Semáforo
self.semaforo = 'VERDE'
self.speed_factor = 1.0

```

Imagen 4. Variables de Configuración y Estado Interno

Para la publicación del objetivo se maneja un mensaje personalizado llamado TargetPose con la siguiente estructura:

```

geometry_msgs/Pose current_goal
geometry_msgs/Pose next_goal

float64 velocity_lin
float64 velocity_ang
bool is_reachable

```

Imagen 5. Mensaje TargetPose

Con esta estructura del mensaje, la lógica del cálculo de objetivos se lleva en `publish_next_goal`, la cual se ejecuta a 100 Hz y decide qué hacer en cada instante.

```

if self.current_target_idx ≥ len(self.targets):
    if not self.mission_finished:
        self.stop_robot()
        self.get_logger().info("¡TRAYECTORIA COMPLETADA!")
        self.mission_finished = True
    return

if self.speed_factor == 0.0:
    self.stop_robot()
    return

```

Imagen 6. Revisiones de Seguridad: objetivo alcanzado y color rojo detectado

Si el robot puede avanzar, toma el objetivo actual y calcula la distancia en línea recta entre dónde está ahorita y a dónde quiere ir.

- `target_angle`: Calcula el ángulo exacto al que debería estar apuntando para mirar directamente al objetivo.
- `error_theta`: Es la diferencia entre hacia dónde está mirando ahora y hacia dónde debería mirar.

```

dx = target['x'] - self.robot_pose['x']
dy = target['y'] - self.robot_pose['y']
dist = np.sqrt(dx**2 + dy**2)
target_angle = np.arctan2(dy, dx)

error_theta = np.arctan2(
    np.sin(target_angle - self.robot_pose['theta']),
    np.cos(target_angle - self.robot_pose['theta'])
)

if dist < self.dist_tolerance:
    self.get_logger().info(f"Punto {self.current_target_idx + 1} alcanzado")
    self.current_target_idx += 1
    return

```

Imagen 7. Cálculo de errores

Posteriormente se usa un valor proporcional (k_{linear} y k_{angular}) el cual, entre mayor sea la distancia o el error angular, más potencia solicitará a los motores y sigue la siguiente logica:

- Giro antes de avance: Si el robot está apuntando en una orientación incorrecta (mayor a ~ 30 grados o 0.52 radianes), decide no avanzar (velocidad lineal = 0) y solo gira hacia el objetivo.
- Avance y giro: Si el robot tiene una orientación cercana a la final, avanza hacia el objetivo mientras corrige el ángulo al mismo tiempo.

```
k_linear, k_angular = 0.8, 2.0

if abs(error_theta) > 0.52:
    msg.velocity_lin = 0.0
    msg.velocity_ang = (k_angular * error_theta) * self.speed_factor
else:
    msg.velocity_lin = (k_linear * dist) * self.speed_factor
    msg.velocity_ang = (k_angular * error_theta) * self.speed_factor
```

Imagen 8. Cálculo de Velocidades

Controller

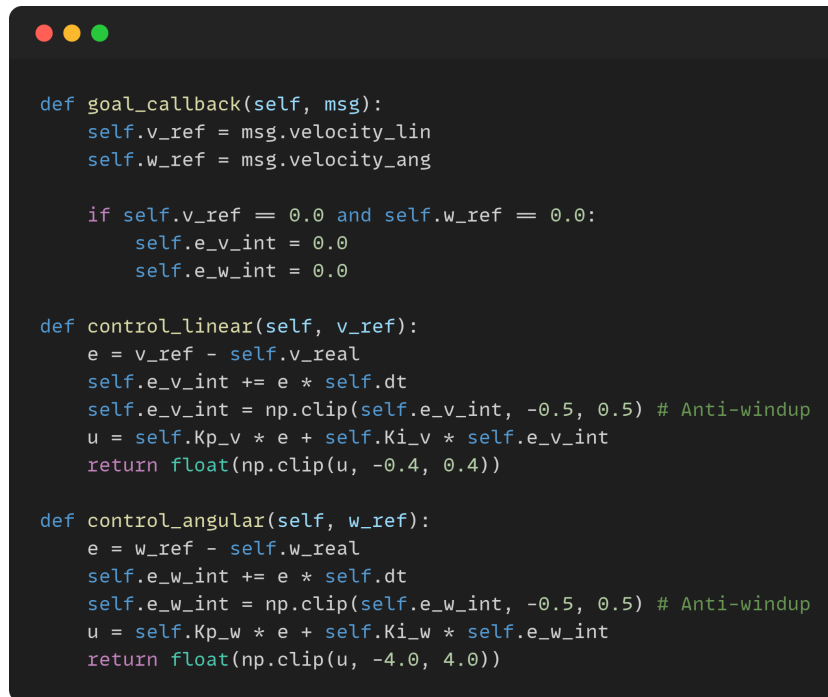
El nodo de control en lazo cerrado tiene como objetivo garantizar el seguimiento preciso de las velocidades comandadas por el generador de trayectorias. Para compensar perturbaciones externas, variaciones en la fricción del suelo y las dinámicas no modeladas del robot, para ello, se implementó un controlador de tipo Proporcional-Integral (PI) que opera de manera independiente para la velocidad lineal y la velocidad angular.

```
# Variables de estado
self.v_real = 0.0
self.w_real = 0.0
self.v_ref = 0.0
self.w_ref = 0.0

# Parámetros PI
self.dt = 0.01
self.Kp_v, self.Ki_v = 1.1, 0.35
self.Kp_w, self.Ki_w = 1.3, 0.35
self.e_v_int, self.e_w_int = 0.0, 0.0
```

Imagen 9. Definición de variables

El controlador basa su funcionamiento en la comparación continua de dos estados principales: las velocidades deseadas publicadas por el nodo de toma de decisiones y las velocidades físicas actuales estimadas por la odometría del robot.



```
def goal_callback(self, msg):
    self.v_ref = msg.velocity_lin
    self.w_ref = msg.velocity_ang

    if self.v_ref == 0.0 and self.w_ref == 0.0:
        self.e_v_int = 0.0
        self.e_w_int = 0.0

def control_linear(self, v_ref):
    e = v_ref - self.v_real
    self.e_v_int += e * self.dt
    self.e_v_int = np.clip(self.e_v_int, -0.5, 0.5) # Anti-windup
    u = self.Kp_v * e + self.Ki_v * self.e_v_int
    return float(np.clip(u, -0.4, 0.4))

def control_angular(self, w_ref):
    e = w_ref - self.w_real
    self.e_w_int += e * self.dt
    self.e_w_int = np.clip(self.e_w_int, -0.5, 0.5) # Anti-windup
    u = self.Kp_w * e + self.Ki_w * self.e_w_int
    return float(np.clip(u, -4.0, 4.0))
```

Imagen 10. Implementación del Control PI

Para evitar comportamientos inestables inherentes al control integral y a las limitaciones físicas de los actuadores, el algoritmo incorpora tres mecanismos de seguridad críticos:

- **Saturación de Salida:** Las señales enviadas a los motores se limitan numéricamente a través de la función `np.clip` para respetar las restricciones físicas de los actuadores, estableciendo una velocidad lineal máxima de 0.4 m/s y una angular de 4.0 rad/s.
- **Protección Anti-windup:** Para prevenir el fenómeno de saturación integral (windup) que ocurre cuando el error se acumula, el término integral acumulado se restringe a un umbral máximo de 0.5. Esto garantiza una respuesta rápida del sistema cuando el error cambia de signo.
- **Reinicio Condicional del Integrador:** En la función de recepción de objetivos se implementó una condición lógica que reinicia los acumuladores integrales a 0.0 cuando se recibe una orden de velocidad nula. Esto elimina cualquier energía residual en el controlador, asegurando que el robot frene de manera estática y no presente movimientos espurios al detenerse frente a un semáforo en rojo.

Estado_Semaforo

El nodo de estado_semaforo tiene la función de procesar el flujo de video en tiempo real de la cámara del robot, aislar las señales luminosas del semáforo mediante técnicas de segmentación de color y publicar el estado detectado para alimentar la lógica de control.

Para garantizar la estabilidad del sistema y evitar la saturación computacional, el nodo implementa un mecanismo de exclusión mutua (Lock). Esto asegura que el algoritmo

procese estrictamente un frame a la vez; si se recibe una nueva imagen mientras la actual sigue en procesamiento, esta se descarta.

```
# Candado para asegurar que solo se procese un frame a la vez
self.lock = Lock()

# Nombre de ventana fijo y creación única
self.window_name = "Deteccion_Semaforo_ROI"
cv2.namedWindow(self.window_name, cv2.WINDOW_AUTOSIZE)

self.config_hsv = {
    'rojo': [((0, 100, 100), (10, 255, 255)),
             ((160, 100, 100), (180, 255, 255))],
    'amarillo': [((10, 100, 100), (45, 255, 255))],
    'verde': [((46, 100, 100), (80, 255, 255))]
}

self.p_ancho = 0.45
self.p_alto = 0.95
self.MIN_AREA = 170
self.MAX_AREA = 1000
```

Imagen 11. Configuración y límites de la detección de color

Una vez adquirida la imagen mediante `cv_bridge` (que traduce el mensaje de ROS a un formato matricial de OpenCV), el frame se redimensiona a una resolución baja (320x240 píxeles). Posteriormente, se extrae una Región de Interés (ROI) central, configurada empíricamente para abarcar el 45% del ancho y el 95% del alto de la imagen.

```
def obtener_roi(self, frame):
    alto, ancho = frame.shape[:2]
    centro_x, centro_y = ancho // 2, alto // 2
    ancho_roi, alto_roi = int(ancho * self.p_ancho), int(alto * self.p_alto)

    x1, y1 = max(0, centro_x - (ancho_roi // 2)), max(0, centro_y - (alto_roi // 2))
    x2, y2 = min(ancho, centro_x + (ancho_roi // 2)), min(alto, centro_y + (alto_roi // 2))

    roi = frame[y1:y2, x1:x2]
    frame_visual = frame.copy()
    cv2.rectangle(frame_visual, (x1, y1), (x2, y2), (0, 255, 0), 2)

    return roi, frame_visual, (x1, y1)
```

Imagen 12. Obtención de la ROI

El procesamiento de la ROI no se realiza en el espectro estándar RGB, sino que se convierte al espacio de color HSV (Matiz, Saturación, Valor). Este cambio es crítico para la robustez del sistema, ya que el espacio HSV separa la información cromática de la intensidad luminosa, haciendo que la detección sea más resistente a variaciones en la iluminación ambiental.

El sistema define diccionarios con límites superiores e inferiores preestablecidos para tres estados:

- Rojo: Se definen dos rangos distintos debido a que este color se ubica en los extremos del espectro circular del Matiz (Hue) en OpenCV (0-10 y 160-180).
- Amarillo: Rango delimitado para tonos intermedios (10-45).
- Verde: Rango delimitado para la señal de avance (46-80).

Por cada color evaluado, el algoritmo genera una máscara binaria (cv2.inRange) donde los píxeles blancos representan coincidencias con los rangos HSV. Para discriminar el ruido visual (como pequeños reflejos o superficies lejanas), el sistema extrae los contornos topológicos de la máscara (cv2.findContours).

```
for color, rangos in self.config_hsv.items():
    mask_final = np.zeros(hsv_roi.shape[:2], dtype=np.uint8)
    for lower, upper in rangos:
        mask = cv2.inRange(hsv_roi, np.array(lower), np.array(upper))
        mask_final = cv2.add(mask_final, mask)

    contorno = self.obtener_contorno_valido(mask_final)
    if contorno is not None:
        estado = color.upper()
        c_detectado = contorno
        break

if c_detectado is not None:
    (x, y), radio = cv2.minEnclosingCircle(c_detectado)
    center = (int(x + x1), int(y + y1))
    radius = int(radio)
    cv2.circle(frame_visual, center, radius, (0, 255, 255), 2)
```

Imagen 13. Detección de color

La lógica de validación obedece a las siguientes reglas:

- Se identifica el contorno con la mayor área dentro de la máscara.
- Se evalúa dicha área contra umbrales espaciales estrictos: un área mínima de 170 píxeles y una máxima de 1000 píxeles.
- Si el área del contorno recae fuera de este rango, se descarta por considerarse ruido o una lectura errónea. Si es válida, se confirma la detección del color.

Resultados

El desempeño general del sistema se evaluó en función de su capacidad para cumplir con los objetivos de navegación punto a punto y su respuesta ante los estímulos visuales, obteniendo resultados mixtos que se detallan a continuación.

La evaluación de la robustez del controlador —definida como la capacidad de mantener la trayectoria planificada a pesar de los factores externos— arrojó resultados altamente positivos. Las estrategias implementadas lograron mitigar los principales problemas físicos del entorno:

- **Mitigación de Perturbaciones Externas:** La decisión de priorizar la corrección simultánea de ángulo y avance en lazo cerrado demostró ser efectiva. El controlador logró absorber las perturbaciones físicas generadas por el deslizamiento y la pérdida de tracción de las llantas del Puzzlebot sobre el suelo liso, manteniendo el rumbo hacia el objetivo.
- **Manejo de No Linealidades (Anti-windup):** La aplicación de límites en la acción de control integral operó de manera exitosa frente a los cambios bruscos de la máquina de estados. Al recibir un factor multiplicativo de 0 (luz roja), el sistema evitó el fenómeno de saturación (windup), impidiendo que el error se acumulara de forma indefinida mientras el robot se mantenía en espera.
- **Precisión Final:** En conjunto, los ajustes de sintonización y la corrección de la odometría fueron suficientes para asegurar un error posicional mínimo. El Puzzlebot logró alcanzar las coordenadas deseadas con un alto grado de precisión, reduciendo drásticamente la desviación entre la posición inicial y final de las trayectorias.

Por otro lado, la integración de la capa de toma de decisiones basada en el color del semáforo, aunque funcional en condiciones controladas, demostró vulnerabilidades significativas ante variables del entorno.

- **Sensibilidad a la Iluminación:** Se observó que el sistema de detección basado en el espacio de color HSV es altamente susceptible a los cambios dinámicos de iluminación, variaciones de brillo y reflejos sobre la superficie, lo que en ocasiones provocaba lecturas intermitentes de la señal.
- **Falsos Positivos en el Entorno:** La principal debilidad del método radica en su dependencia exclusiva de la detección de color. Si en el escenario de prueba existe cualquier otro objeto u elemento del entorno con tonalidades rojas, amarillas o verdes, el Puzzlebot tiende a confundirlo con el objetivo principal, desencadenando la ejecución de comandos incorrectos (como detenerse frente a un objeto rojo no relacionado con el semáforo).

Conclusiones

Se satisfizo el requerimiento principal del reto, lograr que el Puzzlebot actúe y modifique su navegación en respuesta a una señal de semáforo simulada, deteniéndose en rojo, disminuyendo su velocidad en amarillo y avanzando en verde.

Sin embargo, el análisis reveló dos limitaciones importantes a considerar para futuras implementaciones:

- Dependencia del Entorno Visual: Los resultados evidencian que depender exclusivamente de umbrales de color es insuficiente para un escenario real. Es imperativo sustituir esta metodología por técnicas de visión computacional más avanzadas (como detectores de características o redes neuronales) que identifiquen la estructura específica del semáforo, evitando confusiones con otros objetos de colores similares.
- Dependencia de la Superficie de Operación: La sintonización actual del control en lazo cerrado (sus ganancias proporcionales e integrales) fue ajustada y optimizada específicamente para compensar el deslizamiento en una superficie lisa. Si el robot fuera operado sobre una superficie más rugosa o con un mayor coeficiente de tracción, esta misma configuración podría provocar sobre impulsos, oscilaciones agresivas o cambios de velocidad abruptos, ya que el esfuerzo de control calculado sería excesivo. Esto indica que la robustez mecánica del sistema está estrechamente ligada a las condiciones de fricción del suelo de prueba.

Referencias

- Manchester Robotics. (2026). Half term challenge: Vision systems in mobile robotics [Presentación de diapositivas]. MCR2.